Linux Troubleshooting — Scenario #4: "I Killed It, But It Keeps Coming Back"

DevOps on-call runbook • Syed Aftab • Ubuntu (WSL) • systemd + cron • Reference for real incidents & interviews

The alert (7:15 AM): web01 is slow again. Last night's on-call engineer killed a process at 100% CPU. This morning it's back.
Root cause found: `report-gen` was a systemd service with `Restart=always` (defeats `kill`), plus a hidden cron "watchdog" in `/etc/cron.d/report-check` that started it every minute (defeats `systemctl stop`).

The lesson: **a process is only the symptom. Find who owns it and who starts it** — otherwise every fix is temporary.

The methodology (same 6-phase flow)

#	Phase	Commands	Goal
1	Locate	`ps aux --sort=-%cpu | head -5`	Find the top CPU process; spot clues (root, no TTY)
2	Reproduce	`sudo kill PID` → wait → `pgrep`	Prove the problem: it comes back with a new PID
3	Identify	`systemctl status PID`, `systemctl cat`, `journalctl --since/--until`, `grep -rn`	Who owns it (systemd) and who else starts it (cron)
4	Fix	comment out cron → `systemctl stop` + `disable`	Remove every restart source, in the right order
5	Verify	`systemctl status`, `pgrep` — after a full minute	Still down after the schedule interval
6	Prevent	fix the design, `mask` / move unit, timers	Make it impossible to come back by accident

SETUP Building the lab (3 parts)

Part 1 — the program (in a real install location this time, not /tmp)

```
sudo tee /usr/local/bin/report-gen > /dev/null <<'EOF'
#!/bin/bash
while true; do :; done
EOF
sudo chmod +x /usr/local/bin/report-gen
ls -l /usr/local/bin/report-gen        # -rwxr-xr-x 1 root root 35 ...
```

Part	Meaning
`sudo tee FILE`	write into FILE as root. `/usr/local/bin` = where admins install programs
`> /dev/null`	tee also echoes to the screen; this hides that copy
`<<'EOF' ... EOF`	here-document: the lines between the markers go into the file
`while true; do :; done`	infinite do-nothing loop → one core at 100%
`chmod +x`	make executable
why not `sudo cat > FILE`?	the > is done by **your** shell (not root) → Permission denied. With `tee`, the program writing the file runs under sudo

Part 2 — the systemd service

```
sudo tee /etc/systemd/system/report-gen.service > /dev/null <<'EOF'
[Unit]
Description=Nightly report generator

[Service]
ExecStart=/usr/local/bin/report-gen
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now report-gen.service
systemctl status report-gen
```

Part	Meaning
`/etc/systemd/system/`	where admins put their own unit files. File name = service name
`[Unit] Description=`	human-readable name shown in status
`[Service] ExecStart=`	the program systemd runs
`Restart=always`	**trap #1**: if the process dies for **any** reason (including kill), start it again
`RestartSec=3`	wait 3 seconds before restarting
`[Install] WantedBy=multi-user.target`	what `enable` hooks into: start at normal boot
`daemon-reload`	systemd re-reads unit files — required after creating/editing one
`enable --now`	`enable` = start at boot (creates a symlink in multi-user.target.wants); `--now` = also start right now

Status clues: `CPU: 9.622s` after `9s ago` = 100% of a core; `CGroup: /system.slice/report-gen.service` = owned by this service.

Part 3 — the hidden cron watchdog

```
systemctl is-active cron                # active
sudo tee /etc/cron.d/report-check > /dev/null <<'EOF'
* * * * * root systemctl is-active --quiet report-gen || systemctl start report-gen
EOF
```

Part	Meaning
`systemctl is-active`	prints one word: active / inactive. Script-friendly
`/etc/cron.d/`	system cron jobs, one file per job. **Not shown by** `crontab -l`
`* * * * *`	every minute (fields: minute, hour, day-of-month, month, day-of-week)
`root`	files in /etc/cron.d have an extra **user** field
`--quiet`	no text output, only the exit code (0 = running)
`||`	OR: run the next command only if the previous one **failed** (opposite of &&)
meaning	**trap #2**: every minute, "if report-gen isn't running, start it"

STEP 1 Locate — top CPU process (on-call mode starts)

```
ps aux --sort=-%cpu | head -5
USER   PID    %CPU %MEM ... TTY STAT START  TIME COMMAND
root   266329 99.9  0.0 ... ?   Rs   07:16  2:44 /bin/bash /usr/local/bin/report-gen
```

Part / column	Meaning
a	processes of **all** users
u	user-oriented format (USER, %CPU, %MEM, START...)
x	include processes with **no terminal** (services, daemons)
`--sort=-%cpu`	sort by CPU, `-` = highest first
`| head -5`	header + top 4. Must be `head -5` or `head -n 5` (no space after -)
`USER root`	run by root, not a person's account
`TTY ?`	**no terminal** — nobody typed it. Typical of a service or cron job
`STAT Rs`	R = running; s = session leader (daemons start their own session)

Clue vs. Scenario #3: there the process had your user and a terminal (pts/...). Here `root` + ? already hints that **the system** started it, not a person. `ps aux` uses old BSD-style options with no dash.

STEP 2 Reproduce — do what last night's engineer did

```
sudo kill 266329          # sudo: you can't signal another user's (root's) process
ps aux --sort=-%cpu | head -5     # gone! (checked within 3 s)
...wait...
pgrep -a report-gen
267483 /bin/bash /usr/local/bin/report-gen     # BACK, new PID
```

Lesson: "Gone for 3 seconds" is not fixed — that's exactly how last night's engineer closed the ticket. A **new PID** means it didn't survive; something **started a fresh copy**. Killing it again is pointless — find what starts it.

STEP 3 Identify — who owns it, and who else starts it

3a. Which unit owns this PID?

```
systemctl status 267483
* report-gen.service - Nightly report generator
   Loaded: loaded (/etc/systemd/system/report-gen.service; enabled; preset: enabled)
   CGroup: /system.slice/report-gen.service
07:21:19 report-gen.service: Consumed 4min 25.199s CPU time over 4min 25.264s wall clock
07:21:22 report-gen.service: Scheduled restart job, restart counter is at 1.
07:21:22 Started report-gen.service - Nightly report generator.
```

Clue	Meaning
`systemctl status PID`	give a **PID** instead of a name → systemd shows which unit it belongs to
`CGroup: /system.slice/...service`	system service. `user@1000.service`/`session-N.scope` = a person; `cron.service` = cron
`Consumed 4min 25s CPU over 4min 25s wall clock`	CPU time ≈ real time = 100% of a core the whole time
`Scheduled restart job, restart counter is at 1`	systemd restarted it **on purpose** — killed at :19, back at :22 (3 s)

3b. Why does systemd restart it?

```
systemctl cat report-gen        # prints the unit file systemd is actually using (+ overrides)
```

`Restart=always` + `RestartSec=3` → why kill failed. **Design smell:** a "*Nightly* report generator" should run once and exit (timer/cron), not be an always-on service that restarts forever. `Restart=always` suits nginx or a database — for a buggy loop it creates an unkillable process.

3c. Stopped and disabled it — it came back anyway

```
sudo systemctl stop report-gen
sudo systemctl disable report-gen     # Removed '.../multi-user.target.wants/report-gen.service'
...wait...
systemctl status report-gen
   Loaded: ... disabled
   Active: active (running) since 07:26:01        # only "Started", no "restart counter"
```

Clue	Meaning
`disabled` but running	`disable` only affects **boot**. Anything can still run `systemctl start`
no "restart counter" line	not systemd's Restart= this time — **something else asked systemd to start it**
`07:26:01`	first second of a new minute → almost always a **scheduled job** (cron / timer)

3d. Who started it at 07:26:00? — the whole system journal, not one unit

```
journalctl --since "07:25:55" --until "07:26:05"
07:26:01 CRON[268765]: pam_unix(cron:session): session opened for user root
07:26:01 CRON[268767]: (root) CMD (systemctl is-active --quiet report-gen || systemctl start rep...)
07:26:01 systemd[1]: Started report-gen.service - Nightly report generator.
07:26:01 CRON[268765]: pam_unix(cron:session): session closed for user root
```

Part	Meaning
`journalctl`	read the systemd journal — logs from every service, cron included
`--since` / `--until`	time window (today assumed). Use `sudo` if nothing shows
no `-u report-gen`	on purpose: the culprit is **another** program, so read everything in the window
`CRON ... (root) CMD (...)`	**caught in the act**: cron ran the exact command that started the service

3e. Where is the cron job defined?

```
crontab -l          # only: 0 3 * * 0 docker system prune -f  (your real job)
sudo crontab -l     # no crontab for root   <-- but the log said (root)!
sudo grep -rn report-gen /etc/crontab /etc/cron* /var/spool/cron
/etc/cron.d/report-check:1:* * * * * root systemctl is-active --quiet \
    report-gen || systemctl start report-gen
```

Location	What lives there	Shown by crontab -l?
`/var/spool/cron/crontabs/`	each user's personal crontab	Yes (that user's)
`/etc/crontab`	main system crontab	No
`/etc/cron.d/`	one file per job (packages / admins)	**No — the trap**
`/etc/cron.hourly|daily|weekly|monthly/`	scripts on those schedules	No

Part	Meaning
`grep -r`	recursive: search inside folders
`-n`	show line numbers

Part	Meaning
`/etc/cron*`	glob: matches /etc/cron.d, /etc/cron.daily, etc.
`crontab -l` / `sudo crontab -l`	list your / root's personal cron jobs

STEP 4 Fix — remove every restart source, in the right order

Order matters: disable the watchdog **first**, then stop the service. Otherwise cron can start it again in between.

```
sudo sed -i 's/^/# DISABLED 2026-10-28 high CPU incident: /' /etc/cron.d/report-check
cat /etc/cron.d/report-check
# DISABLED 2026-10-28 high CPU incident: * * * * * root systemctl is-active ...
sudo systemctl stop report-gen
```

Part	Meaning
`sed -i`	edit in place
`s/^/TEXT/`	`^` = start of each line → adds TEXT to the front of every line
`# DISABLED <date> <reason>`	# makes cron ignore the line; the note tells the next engineer why and when
comment, not `rm`	reversible — the original job is preserved if the owner needs it
`systemctl stop` vs `kill`	a manual stop is respected: `Restart=` does not apply. `kill` looks like a crash → restarted

What we did to disable it — summary

Restart source	What it defeated	How we disabled it
systemd `Restart=always`	`kill PID` (back in 3 s)	`systemctl stop` — a manual stop is never auto-restarted
systemd `enable` (boot)	a reboot	`systemctl disable` — removed the multi-user.target.wants symlink
cron `/etc/cron.d/report-check`	`systemctl stop` (back at the next minute)	commented out with `sed` + dated note

STEP 5 Verify — across at least one full schedule interval

```
systemctl status report-gen       # checked at 07:34, after stop at 07:32:37
o report-gen.service - Nightly report generator
   Loaded: loaded (...; disabled; preset: enabled)
   Active: inactive (dead)
pgrep -a report-gen               # nothing, exit code 1
```

Clue	Meaning
o hollow circle	inactive (filled green dot = running)
`disabled`	won't start at boot
`preset: enabled`	only the distro's default policy for new units — doesn't override your disable
07:33:00 and 07:34:00 passed	two cron runs passed with no restart → **fixed**

Incident timeline from the status log:

Time	Event
07:21:22	restarted by systemd after `kill` (Restart=always)
07:25:47	stopped with systemctl stop (had used 4m24s CPU in 4m27s)
07:26:01	**started again by cron** (/etc/cron.d/report-check)
07:32:37	stopped after the cron job was disabled (6m36s CPU in 6m36s)
07:33 →	stayed down

STEP 6 Prevent

1. Lock it during an incident — `mask` (and when it doesn't work)

```
sudo systemctl mask report-gen
Failed to mask unit: File '/etc/systemd/system/report-gen.service' already exists.
```

Unit file location	Put there by	mask?
`/usr/lib/systemd/system/`	a package (`apt install nginx`)	Yes — mask link in /etc/systemd/system overrides it
`/etc/systemd/system/`	an admin (our lab)	**No** — that path is already the real file

`mask` links the unit to `/dev/null` so it can't be started from anywhere (person, cron, boot). Undo with `unmask`. For an admin-made unit, move the file out instead:

```
sudo mv /etc/systemd/system/report-gen.service /root/report-gen.service.disabled
sudo systemctl daemon-reload        # now: Unit report-gen.service not found
```

2. Fix the design (real root cause): a nightly job should be `Type=oneshot` (run once, exit) triggered by a **systemd timer** (check with `systemctl list-timers`) or one cron line — with `flock -n` and `timeout` (Scenario #3). No `Restart=always`, no watchdog.

3. Ask the owner before deleting. That's why the cron line was commented with a dated note, not removed.

4. Verify across a full schedule interval — at least one minute for a per-minute cron, longer for hourly jobs.

Lab cleanup

```
sudo rm /etc/systemd/system/report-gen.service
sudo rm /etc/cron.d/report-check
sudo rm /usr/local/bin/report-gen
sudo systemctl daemon-reload
systemctl status report-gen        # Unit report-gen.service could not be found.
```

REF Who owns it? — the decision table

Owner (from systemctl status PID / parent)	Stop it now	Stop it coming back
a person (session / terminal)	`kill PID`	ask who ran it
cron (`cron.service`, parent `cron`)	`kill PID`	comment out the line (crontab -e or /etc/cron.d)
systemd service	`systemctl stop NAME`	`systemctl disable NAME`; `mask` or move unit file
systemd timer	`systemctl stop NAME`	`systemctl disable --now NAME.timer`
service + cron watchdog (this lab)	disable cron first, then `stop`	`disable` + comment out cron

REF Full command reference

Command	What	Why here
`ps aux --sort=-%cpu | head -5`	top CPU processes, all users, incl. daemons	locate; root + no TTY clue
`sudo kill PID`	SIGTERM a root-owned process	reproduce: it comes back
`pgrep -a NAME`	find by name + full command	compare PIDs before/after
`systemctl status PID`	which unit owns a process	owner: service, cron or person
`systemctl cat NAME`	show the unit file in use	found Restart=always
`systemctl stop / disable NAME`	stop now / don't start at boot	the systemd fix
`systemctl enable --now NAME`	start at boot + now	lab setup
`systemctl is-active NAME`	one-word state	script-friendly check
`systemctl daemon-reload`	re-read unit files	after creating/removing units
`systemctl mask / unmask`	make a unit unstartable / undo	lock during incident (package units)
`systemctl list-timers`	scheduled systemd timers	the other place jobs hide
`journalctl --since "T1" --until "T2"`	all logs in a time window	caught CRON starting it
`journalctl -u NAME`	logs for one unit	restart history
`crontab -l` / `sudo crontab -l`	your / root's personal cron jobs	first place to look
`sudo grep -rn TEXT /etc/crontab /etc/cron* /var/spool/cron`	search every cron location	found /etc/cron.d/report-check
`sed -i 's/^/# NOTE /' FILE`	comment out every line	disable cron job, reversibly
`sudo tee FILE <<'EOF'`	write a root-owned file	lab setup

How to remember all this

Don't memorize commands — memorize the **questions**, in order. Each question has one or two go-to commands:
1. **Is it really broken?** → uptime, curl, df -h
2. **What exactly is wrong?** → top / ps aux --sort, systemctl status
3. **Why? What do the logs say?** → journalctl (-u, --since), nginx -t, the error message
4. **Who owns it / who starts it?** → systemctl status PID, ps -o ppid, crontab, grep -r
5. **Fix safely** → backup, test, the gentlest tool first
6. **Did it stay fixed?** → check again after time passes

Flags you look up (`man ps`, `CMD --help`, your runbooks). The questions are the skill.

Interview answer (say this)

"If a process keeps coming back after kill, killing it again is pointless — something is restarting it. I check a new PID appeared, then systemctl status on the PID to see which unit owns it, and systemctl cat to see Restart=always. I stop and disable the service rather than kill it. If it still returns, I look at the start time — on the minute means a scheduler — and read journalctl for that window, which shows CRON running systemctl start. crontab -l won't show /etc/cron.d, so I grep every cron location, comment the job out with a dated note, then stop the service, and verify it stays down across a full interval. Long term: a nightly job should be a oneshot with a timer, not an always-restarting service with a watchdog."